



REPO2ROCM: An Agentic System for Reproducing Research Code on AMD ROCm GPUs

EMNLP 2026
INDUSTRY TRACK

BUDAPEST · OCT 24–29, 2026

Gaurav Sarkar^{1†} Umang Pandey² Pratik Prabhanjan Brahma² Zicheng Liu² Emad Barsoum²

¹IIIT Hyderabad ²Advanced Micro Devices, Inc. (AMD) [†]Work done while at AMD

WHY REPO2ROCM?

Research code accompanying many ML papers is developed in CUDA-based environments. Reproducing it on AMD ROCm requires environment repair, kernel migration and runtime **proof that the intended GPU workload is preserved**.

59/60

repos verified in Mode 1

3.3× vs. 18/60 in Mode 0 (out of the box)

77.3

Portability Fidelity Score

vs. 61.4 for Claude Code · same LLM

18–20 min

GitHub URL → verified GPU run

zero manual intervention

1 · THE PROBLEM: “IT RUNS” IS NOT “IT REPRODUCES”

42 of 60 recent research repos fail out of the box (Mode 0)

Tier 1 · Research-code reproducibility

Reconstruct the setup

Stale instructions, dependency conflicts, version mismatches and outdated APIs.

Recover the intended experiment

Unclear entry points, missing datasets or checkpoints, and author-specific scripts.

Tier 2 · CUDA to ROCm migration

Choose a compatible ROCm stack

Match containers, frameworks and packages to the repository and the target AMD GPU.

Preserve GPU libraries and device logic

Adapt FlashAttention/vLLM builds and hardware-specific device assumptions.

Port custom CUDA kernels

.cu files and extensions need CUDA-to-HIP conversion, then build-and-load validation.

Reject silent loss of fidelity

Detect CPU fallback, disabled components, degraded backends and toy-scale workloads.

Not enough: Install, import or device checks.

Required: Real AMD-GPU workload; GPU path intact.

WHAT WE CONTRIBUTE

1 · SYSTEM

Repo2ROCM

An agentic system that migrates CUDA-centric research repos to AMD ROCm and proves a real GPU run, with zero manual intervention.

3 · BENCHMARK

ResearchBench

60 real repos from recent ICLR, ICML, ACL and NeurIPS papers: no curated issues or tests, stale setup, unclear entry points. Only 18/60 pass in Mode 0.

2 · METHOD

Evidence-gated plan-then-execute

Plan statically outside Docker, then act under hard guards: no environment change, source edit, or success claim without deterministic runtime evidence.

4 · METRIC

Portability Fidelity Score (PFS)

0–100 score for faithful execution: accelerator path kept, real workload run, ROCm environment verified. Repo2ROCM 77.3 vs. 61.4 for Claude Code.

2 · METHOD: PLAN FIRST, THEN EXECUTE UNDER EVIDENCE GUARDS

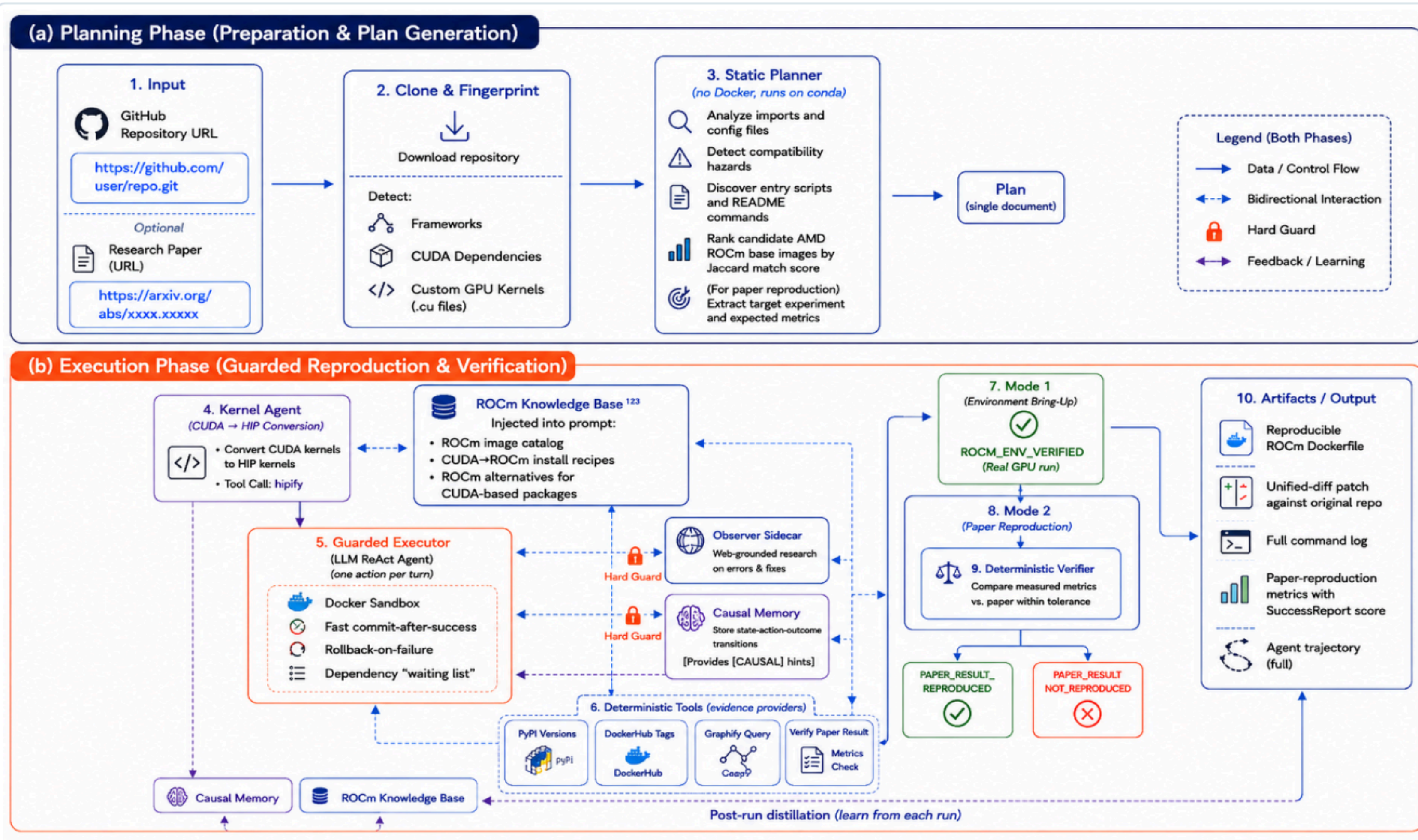


Fig. 1 Plan outside Docker → guarded execution in a GPU-pinned Docker sandbox → Mode 1 verification (optional Mode 2 metric check).

What counts as success

The three settings behind every result

MODE 0 Out of the box

18/60

30.0%

Clone the repo and follow its public README on the AMD GPU; Repo2ROCM does not intervene. Success: the official workload runs GPU-backed, as-is.

vs.

MODE 1 Functional reproduction

59/60

98.3% · main claim

Repo2ROCM rebuilds a ROCm environment, adapts CUDA-centric dependencies, kernels and device checks, and runs an official workload. Success: it completes on the AMD GPU with the accelerator path intact.

ROCm ENV VERIFIED

then

MODE 2 Paper-metric check

1 of 2

case studies matched

After verified Mode 1 execution, compare the selected command's metrics against paper targets and specified tolerances.

TurboQuant: selected quantization results reproduced (case study below).

Preliminary Mode 2 evaluation; one of two cases matched.

3 · WHAT'S DIFFERENT: NO EVIDENCE, NO ACTION

Not a new component, a contract: every irreversible action and every success claim must be backed by runtime evidence the agent cannot override.

Same failure, two outcomes

General-purpose coding agent

FlashAttention (CUDA build) won't load on ROCm → Swap in PyTorch SDPA → “It runs!” → Degraded path: not the paper's method

Repo2ROCM

FlashAttention (CUDA build) won't load on ROCm → Hard guard: a backend swap needs evidence → ROCm FlashAttention (e.g., AMD Triton) + GPU check → ROCm ENV VERIFIED

Observed in agent runs: FlashAttention → SDPA (Knowledge-Entropy, PrefixKV) · altered call signature (PrefixKV) · module left disabled (UniZyme).

Both runs “work”; only one still evaluates the paper's method.

Why this design works

1 Plan outside Docker

Jaccard-ranked ROCm base images; cheap discovery before any container turn.

2 Hard guards

No image switch, library swap, critical edit, or success claim without evidence.

3 Checkpoint + rollback

Failed attempts never corrupt the last known-good environment.

4 Non-overridable verifier

An LLM can narrate success for any run; runtime evidence decides.

How we score it: PFS (0–100)

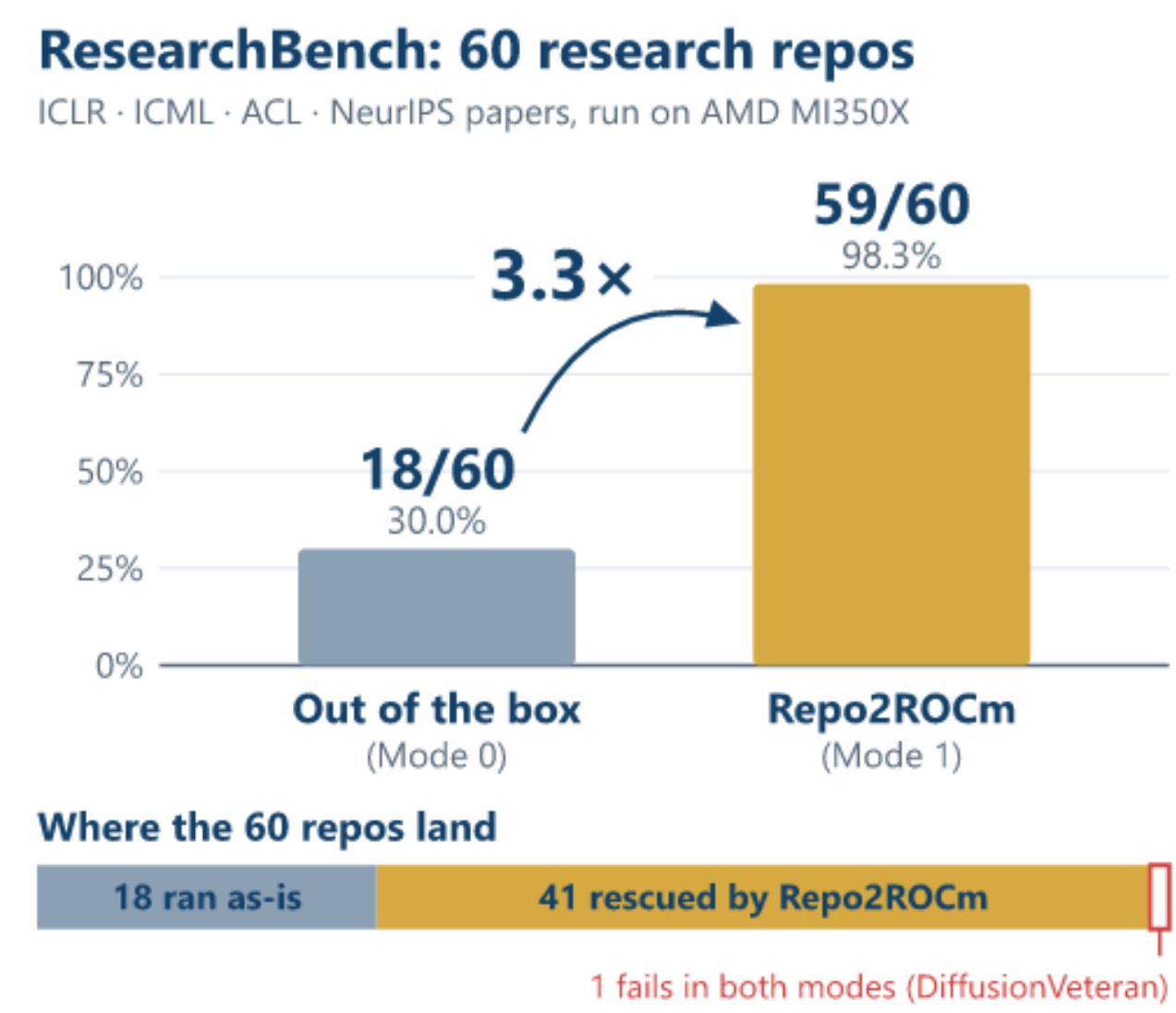
$$PFS = \text{Ran} \times (15 + 35 \cdot F_{\text{path}} + 20 \cdot A_{\text{work}} + 20 \cdot A_{\text{env}} + 10 \cdot D_{\text{proc}})$$

35	path fidelity	
20	real workload	
20	verified ROCm env	
10	process discipline	
15	ran (floor)	

Path grade per accelerator-critical component: 1.0 preserved · 0.5 degraded (SDPA, CPU fallback) · 0 disabled or signature-altered.

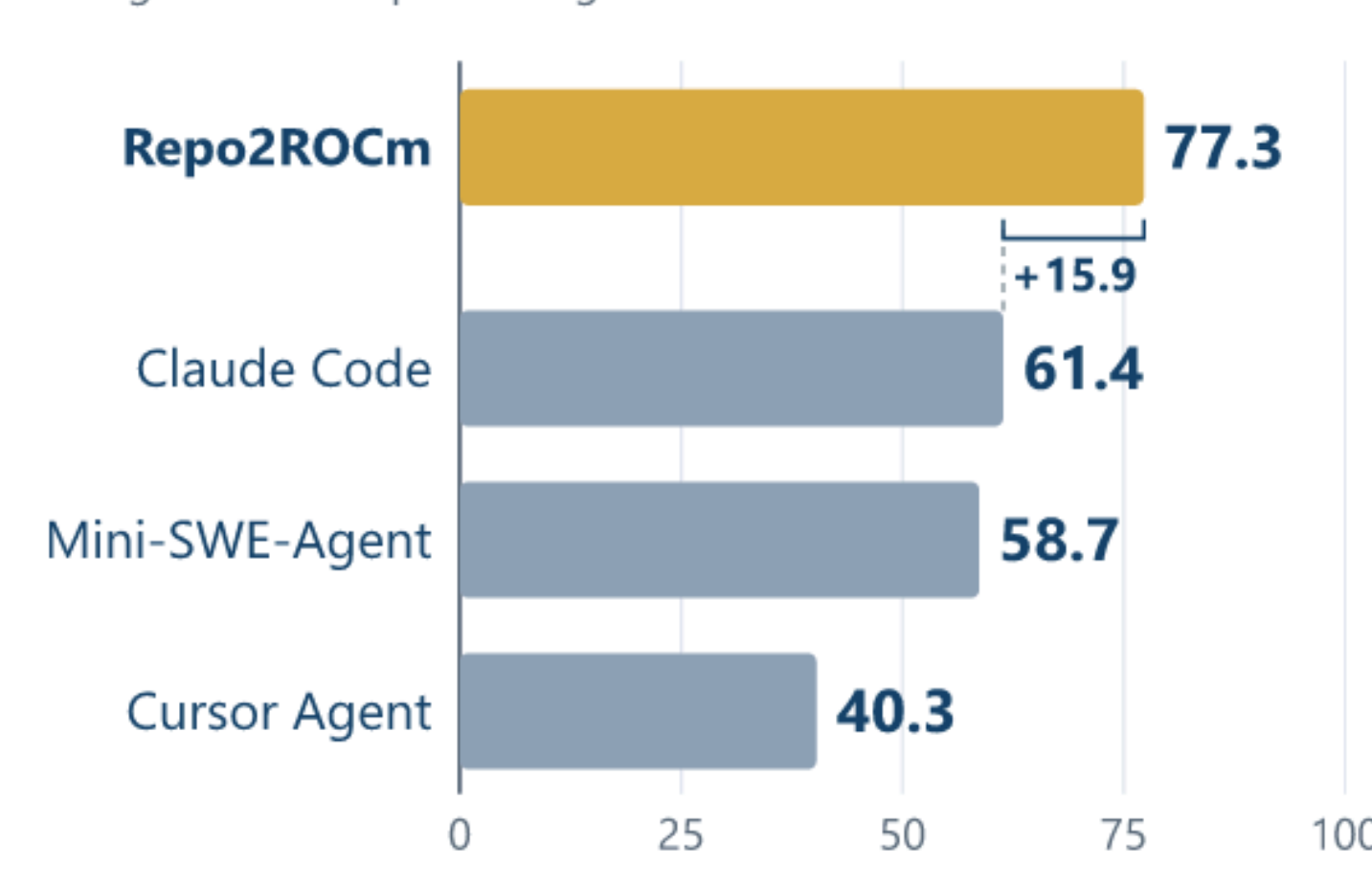
One LLM judge (Claude Opus 4.6), same frozen rubric for all agents.

4 · RESULTS: 3.3× MORE REPOS REPRODUCED, WITH THE HIGHEST FIDELITY



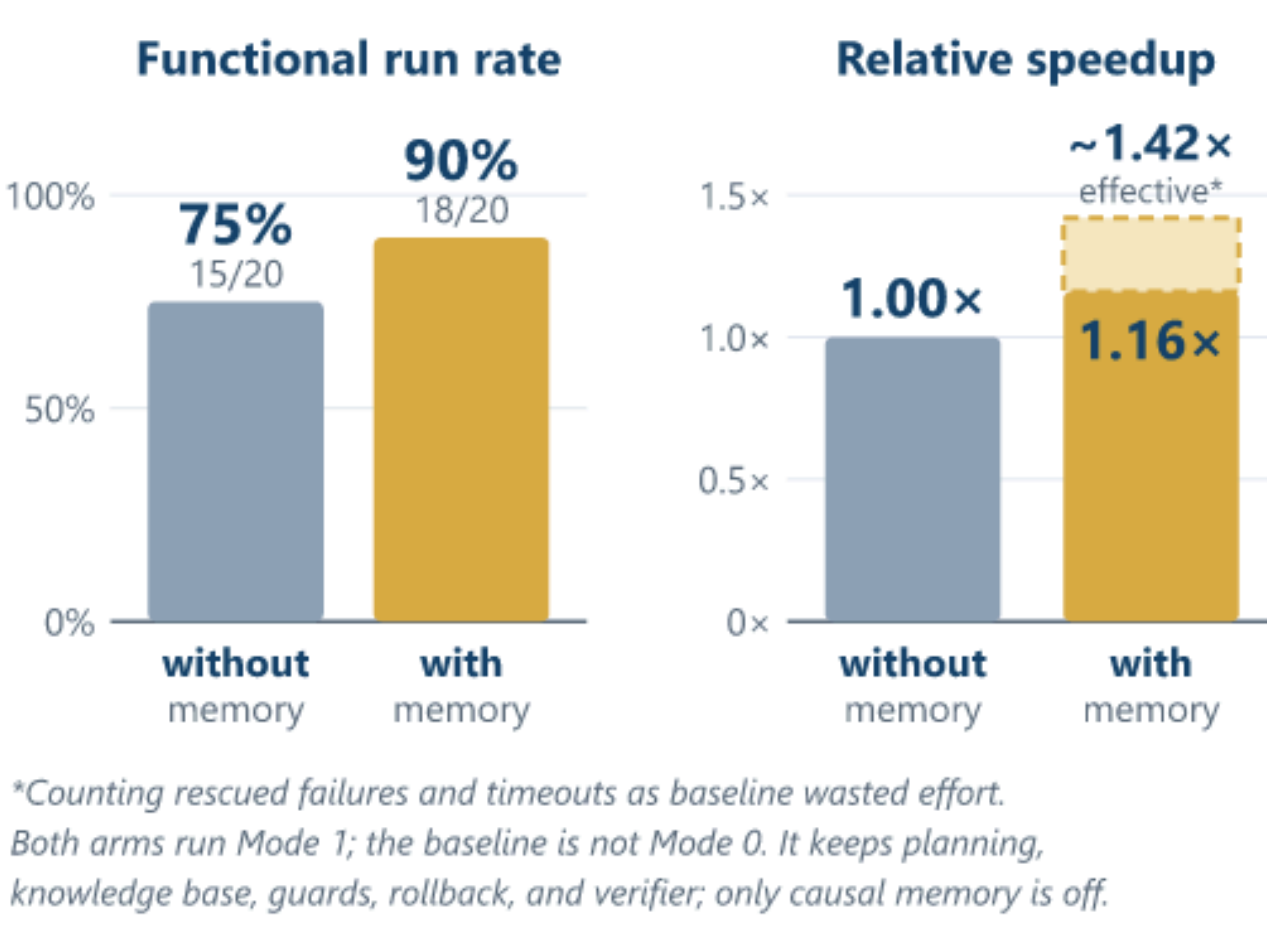
Fidelity vs. general coding agents

Average PFS · 10 repos · all agents on Claude Sonnet 4



Causal memory: more repos, faster

Paired ablation · 20 CUDA-oriented, GPU-dependent repos



TurboQuant: verified quantization

Automated workflow · Mode 2 · ICLR 2026

- Identify a checkable experiment**
Identify the quantization command, target metrics and tolerances from the paper/code.
- Prepare and run on AMD ROCm**
Reconstruct the environment, run the workload, and verify AMD-GPU execution (Mode 1).
- Check the selected paper results**
Compare measured metrics with the reported targets and specified tolerances.

ALL SELECTED METRICS WITHIN TOLERANCE
✔ Compression ✔ Similarity ✔ Match rate
PAPER_RESULT_REPRODUCED

Selected metrics only · Preliminary Mode 2 · Paper §§2, 3.3

All runs: 8× AMD Instinct MI350X (gfx950), ROCm 1.18 · Claude Sonnet 4 backbone, ≤100 turns · ≈719K tokens (median 608K) and 18–20 min per repo · 15/15 Mode 1 reruns gave the same verdict

TAKEAWAY + LINKS

What to remember

For industry teams: a GitHub URL in, a verified AMD-GPU run out (Mode 1) in ≈18–20 min, with zero manual steps.

The edge is evidence-gated migration: no irreversible change and no success claim without runtime proof.

Next: scale Mode 2 from two case studies to full paper-result reproduction.

Every run ships an audit bundle: Dockerfile · source patch · command logs · verification evidence · agent trajectory



Code

github.com/Gaurav7888/Repo2ROCM

Scan the QR code for code & artifacts

REPO2ROCM · EMNLP 2026 Industry Track · Budapest

References: ReAct · Reflexion · SWE-bench · SWE-agent · Repo2Run · FlashAttention · vLLM · AMD ROCm